

Relatorio Tecnico de Resiliencia

1. Visao geral

Este trabalho implementa uma aplicacao distribuida de pedidos usando Spring Boot, PostgreSQL e Redis em Kubernetes. A API usa Redis para idempotencia, PostgreSQL para persistencia dos pedidos e mecanismos de tolerancia a falhas para degradar o fluxo de pagamento de forma controlada.

2. Arquitetura

Componentes:

- API `orders-api`: recebe pedidos, aplica idempotencia e registra o estado final.
- Banco `postgres-service`: persiste pedidos.
- Cache `redis-service`: armazena chaves de idempotencia.

Mecanismos de mitigacao:

- `readinessProbe` e `livenessProbe` para recuperacao automatica pelo Kubernetes.
- Duas replicas minimas da API e HPA ate cinco replicas.
- Requests/limits de CPU e memoria.
- Timeout HTTP, Retry e Circuit Breaker via Resilience4j.
- Fallback do pagamento remoto para pedido `FAILED`, evitando erro HTTP 500 quando a dependencia externa falha.
- Metricas em `/actuator/prometheus`.

3. Estado estavel esperado

Antes dos experimentos:

- Pods `orders-api`, `postgres` e `redis` em estado `Running`.
- Endpoint `/actuator/health` com status `UP`.
- Criacao de pedidos respondendo HTTP `201`.
- Consulta `/orders` respondendo HTTP `200`.
- HPA mantendo duas replicas quando a carga esta baixa.

Comandos de coleta:

```
kubectl get pods
kubectl get hpa
kubectl logs deploy/orders-api-deployment --tail=100
Invoke-RestMethod http://localhost:30080/actuator/health
Invoke-RestMethod http://localhost:30080/actuator/prometheus
```

4. Experimento 1 - Falha de rede

Manifesto: `k8s/chaos/network-latency.yaml`

Estado estavel:

- API acessa PostgreSQL com latencia normal.
- Criacao e consulta de pedidos respondem dentro do tempo esperado.

Hipotese:

- A latencia de 500 ms entre API e PostgreSQL aumenta o tempo de resposta.
- As probes continuam saudaveis se a aplicacao ainda conseguir responder.
- A API deve continuar funcional, possivelmente mais lenta.

Configuracao do ataque:

- Tipo: `NetworkChaos`.
- Acao: `delay`.
- Origem: pods com label `app=orders-api`.
- Destino: pods com label `app=postgres`.
- Latencia: `500ms`.
- Duracao: `5m`.

Resultado observado:

- Preencher durante a demonstracao com tempo medio de resposta, prints de logs e metricas.

Acoes corretivas:

- Probes de saude para remover pods indisponiveis do trafego.
- Requests/limits para previsibilidade.
- Metricas Prometheus para comparar estado estavel e estado sob falha.

5. Experimento 2 - Falha de instancia

Manifesto: `k8s/chaos/pod-kill.yaml`

Estado estavel:

- Duas replicas da API em execucao.
- Service `orders-api-service` balanceando trafego entre replicas.

Hipotese:

- Ao matar um pod da API, o Service redireciona trafego para a replica restante.
- O Deployment recria o pod morto.
- Pode ocorrer uma falha transitoria durante requisicoes ativas, mas o sistema recupera automaticamente.

Configuracao do ataque:

- Tipo: `PodChaos`.
- Acao: `pod-kill`.
- Alvo: um pod com label `app=orders-api`.
- Duracao declarada: `30s`.

Resultado observado:

- Preencher durante a demonstracao com `kubectl get pods -w`, logs e status das requisicoes.

Acoes corretivas:

- Deployment com `replicas: 2`.
- Liveness/readiness probes.
- Service Kubernetes desacoplando cliente das instancias.

6. Experimento 3 - Falha de recurso

Manifesto: `k8s/chaos/cpu-stress.yaml`

Estado estavel:

- Uso de CPU abaixo do limite.
- HPA mantendo duas replicas.

Hipotese:

- O stress de CPU degrada a latencia da API.
- O HPA aumenta replicas se o metrics-server estiver instalado e a media ultrapassar 70%.
- Limits impedem consumo descontrolado de CPU.

Configuracao do ataque:

- Tipo: `StressChaos`.
- Alvo: um pod com label `app=orders-api`.
- CPU workers: `2`.
- Load: `90`.
- Duracao: `5m`.

Resultado observado:

- Preencher durante a demonstracao com `kubectl top pods`, `kubectl get hpa -w` e metricas Prometheus.

Acoes corretivas:

- HPA de 2 a 5 replicas.
- Requests/limits definidos no container.
- Endpoint Prometheus para observar impacto.

7. Conclusao

A aplicacao atende aos requisitos de arquitetura distribuida e oferece um conjunto minimo de mitigacoes para falhas de rede, instancia e recurso. Os experimentos Chaos Mesh permitem demonstrar degradacao, recuperacao automatica e observabilidade do sistema durante a apresentacao.